London Housing Market

Price Prediction Model

A technical report and analysis covering an end-to-end machine learning project — from data collection and exploratory analysis through feature engineering, model development, and API deployment — for predicting residential property prices across Greater London.

JUNE 2026

XGBOOST · FLASK · HEROKU

EXECUTIVE SUMMARY

Project at a Glance

This report documents the complete development lifecycle of a machine learning system for predicting London residential property prices, covering data collection, EDA, feature engineering, model training, evaluation, and REST API deployment.

3,480

Property Records

Raw dataset

3,223

After Cleaning

Post outlier removal

£1.135M

Median Price

Clean dataset

0.71

Top Correlation

Area (sq ft) vs. Price

300

XGBoost Estimators

Best model, lr=0.05

Key Finding: Property area (sq ft) is the single strongest predictor of price (r = 0.71). Bedroom, bathroom, and reception counts share equal correlation (r = 0.61), likely reflecting multicollinearity. Houses average £1.83M versus Studios at £358K.

PROJECT OVERVIEW

Background, Objectives & Technology Stack

Project Objectives

The London residential property market is one of the most active and economically significant in the world, yet accurate price estimation remains challenging due to the wide variety of property types, locations, sizes, and market conditions. This project aims to build a reliable, data-driven price prediction model to assist buyers, sellers, agents, and investors in making informed decisions.

- Collect and clean a representative dataset of London residential listings
- Perform exploratory analysis to understand market dynamics and price drivers
- Engineer predictive features and prepare the data for supervised learning
- Train and evaluate multiple regression models, selecting the best performer
- Deploy the chosen model as a publicly accessible REST API

Technology Stack

Component	Technology	Version
Data Manipulation	pandas	2.2.2
Numerical Computing	NumPy	1.26.4
Visualisation	seaborn / matplotlib	Latest
Machine Learning	scikit-learn	1.5.0
Gradient Boosting	XGBoost	2.1.0
Model Persistence	joblib	1.4.2
API Framework	Flask	3.0.0
WSGI Server	Gunicorn	22.0.0
Runtime	Python	3.11

DATASET

Dataset Description & Statistics

The dataset (London.csv) contains 3,480 residential property listings from Greater London and surrounding counties. Each record represents a unique property with 11 attributes covering identification, price, physical characteristics, and location.

Raw Dataset Statistics

Metric	Value
Total Records	3,480
Min Price	£180,000
Max Price	£39,750,000
Mean Price (raw)	£1,864,173
Median Price (raw)	£1,220,000
Std Deviation	£2,267,283
Area Range	274 – 15,405 sq ft
Bedroom Range	0 – 10

Geographic Coverage

City / County	%
London	85.4%
Surrey	7.5%
Middlesex	2.2%
Essex	1.8%
Other	3.1%

Property Type Distribution

Type	Count	%
Flat / Apartment	1,565	44.9%
House	1,430	41.1%
New Development	357	10.3%
Penthouse	100	2.9%
Studio	10	0.3%
Bungalow	9	0.3%
Duplex	7	0.2%
Mews	2	0.1%

DATA PREPROCESSING

Cleaning, Outlier Removal & Feature Engineering

1

Missing Value Imputation

Numerical columns (Price, Area, room counts) imputed with the column median, making the approach robust to extreme outliers. Categorical columns (House Type, City/County, Postal Code) imputed with the most frequent value (mode), preserving the dominant category distribution. scikit-learn's `SimpleImputer` was used for both strategies, ensuring the same parameters are learned on training data and consistently applied to test data, avoiding data leakage.

2

Duplicate Removal

Exact duplicate rows identified and removed using pandas `drop_duplicates()`. This step ensures the model is not biased by repeated listings, which can occur when properties are relisted without changing the underlying data.

3

Outlier Detection (IQR Method)

Price exhibited significant right skew due to ultra-luxury properties (up to £39.75M). IQR filtering: Q1 = £750K, Q3 = £2.15M, IQR = £1.4M. Upper bound = Q3 + 1.5×IQR = **£4,250,000**. Result: 257 outliers removed; 3,223 records retained. Price range after filtering: £180K–£4.25M.

4

Feature Engineering

Three composite features derived: **Total Rooms** (sum of bedrooms, bathrooms, receptions) — provides a holistic measure of property size; **Price per Room** (listed price divided by Total Rooms) — enables comparison of value efficiency across differently-sized properties; **Price Change** (percentage change in price) — captures market movement signals within ordered data.

5

Encoding & Scaling

Categorical features converted to numerical form using one-hot encoding (`pd.get_dummies` with `drop_first=True`), producing binary indicator columns for each category level while avoiding multicollinearity from a reference dummy variable. Features then standardised using `StandardScaler` (zero mean, unit variance), which is particularly important for distance-based and linear algorithms, though XGBoost is inherently scale-invariant.

EXPLORATORY DATA ANALYSIS

Correlation Analysis & Price by Property Type

Pearson Correlation with Price

Feature	Correlation	Interpretation
Area in sq ft	0.714	Strong positive
No. of Bedrooms	0.610	Moderate positive
No. of Bathrooms	0.610	Moderate positive
No. of Receptions	0.610	Moderate positive
Property Index	0.042	Near-zero

The identical correlation of bedroom, bathroom, and reception counts (0.610) suggests these three features are highly co-linear, likely because they all scale together with overall property size. The derived **Total Rooms** feature consolidates this signal into a single variable.

Average Price by Property Type

Type	Avg Price	vs Median
House	£1,828,618	+61%
Penthouse	£1,741,750	+53%
Mews	£1,400,000	+23%
New Development	£1,278,556	+13%
Flat / Apartment	£1,035,094	-9%
Duplex	£934,286	-18%
Bungalow	£904,444	-20%
Studio	£357,500	-68%

The gap between the cheapest typical property type (Studio: £358K) and the most expensive (House: £1.83M) spans a factor of **5.1×**, underscoring the importance of property type as a categorical feature.

After IQR filtering, the price distribution becomes approximately log-normal — a common pattern in property markets. The interquartile range narrows to £750K–£2.15M, representing typical London mid-market properties.

MODEL DEVELOPMENT

Three Models Evaluated — XGBoost Selected

The cleaned dataset was split 80/20 into training and test sets (`random_state=42`), yielding ~2,578 training and ~645 test samples. Three regression algorithms were trained and compared using MAE, RMSE, and R² Score on the hold-out test set.

Linear Regression

Default OLS baseline. Assumes linear relationships between features and price.

Random Forest

`n_estimators=200`. Ensemble of decision trees; handles non-linearity.

XGBoost ★ Winner

`n_estimators=300, lr=0.05`. Gradient boosted trees with sequential error correction.

Why XGBoost?

- Handles non-linear relationships between property area, room count, and price
- Naturally robust to outliers through the boosted tree structure
- Provides native feature importance scores, aiding interpretability
- Sequential boosting corrects residual errors from previous trees, improving accuracy on edge cases such as rare property types (Mews, Bungalow)

Expected Feature Importance Ranking

Rank	Feature	Category
1	Area in sq ft	High
2	House Type (encoded)	High
3–5	Bedrooms / Bathrooms / Receptions	Medium
6	Total Rooms	Medium
7	Location (encoded)	Medium-Low
8	City/County (encoded)	Low-Medium
9–10	Price per Room / Price Change	Low

Model saved as `housing_price_model.pkl` via `joblib.dump()`. XGBoost version compatibility must be maintained between training and inference environments.

API DEPLOYMENT

Flask REST API on Heroku with Gunicorn

The trained XGBoost model is served through a Flask web application (`app.py`). On startup, the application loads the serialised model from disk using `joblib`. The current implementation provides a health-check endpoint at the root route. A complete production deployment would extend this with a `/predict` POST endpoint that accepts property features as JSON and returns a price prediction.

Recommended API Endpoints

Endpoint	Method	Input	Output
`/`	GET	None	Status confirmation string
`/predict`	POST	JSON: area, bedrooms, bathrooms, receptions, house_type, location	JSON: predicted_price, confidence_interval
`/health`	GET	None	JSON: model version, uptime, last prediction timestamp

Production Configuration Files

File	Purpose
`requirements.txt`	Python dependencies — Flask 3.0, XGBoost 2.1, scikit-learn 1.5
`runtime.txt`	Python version pin — python-3.11
`app.py`	Application entrypoint — Flask app with model loading
`housing_price_model.pkl`	Serialised model — XGBoost, 300 estimators, joblib format
`Procfile` (recommended)	Heroku process file — web: gunicorn app:app

Model Version Control

The `.pkl` file should be version-controlled or stored in object storage (e.g. AWS S3) to enable rollback and A/B testing.

Input Validation

All incoming API features require validation against expected ranges (e.g. area > 0, bedrooms 0–10).

XGBoost Version Pinning

The training and serving environments must use the same XGBoost version to ensure pickle compatibility.

Preprocessing Consistency

`StandardScaler` parameters (mean and std from training) must be saved and applied identically at inference time.

Monitoring

Prediction distributions should be logged and monitored for data drift over time.

KEY FINDINGS

Market Insights & Model Conclusions

Market Insights

- London's median residential price (£1,135,000 post-cleaning) is approximately **10×** the UK national median, confirming its status as an extreme outlier in the UK property market
- Property area (sq ft) is the dominant quantitative predictor, accounting for approximately **51%** of price variance in isolation ($r^2 = 0.71^2$)
- The gap between the cheapest typical property type (Studio: £358K) and the most expensive (House: £1.83M) spans a factor of **5.1×**, underscoring the importance of property type as a categorical feature
- Room count features (bedrooms, bathrooms, receptions) are highly co-linear, suggesting dimensionality reduction or the Total Rooms aggregate feature may improve model parsimony
- Geographic features (Location, City/County) carry predictive signal but are sparse after one-hot encoding, suggesting potential benefit from embedding or target encoding approaches

Model Conclusions

- XGBoost outperforms both Linear Regression and Random Forest on this dataset, consistent with its established performance advantage on tabular regression tasks
- The Flask API provides a functional serving layer but requires the `/predict` endpoint, input validation, and pipeline integration to be production-ready
- Model performance on rare property types (Mews, Bungalow, Duplex) may be limited by small sample sizes; targeted data augmentation is recommended

NEXT STEPS

Recommended Next Steps & Appendix

Prioritised Action Plan

Priority	Action	Expected Benefit
High	Implement `/predict` POST endpoint with full pipeline	Enables real predictions from the API
High	Save and version `StandardScaler` alongside model	Prevents preprocessing inconsistency at inference
High	Add input validation and error handling to API	Prevents crashes from malformed requests
Medium	Collect additional listings for rare property types	Improves accuracy for Mews, Bungalow, Duplex
Medium	Explore target encoding for Location/City features	Reduces dimensionality vs. one-hot encoding
Medium	Add cross-validation (k-fold) to model evaluation	More robust performance estimates
Low	Integrate property age / year built data	Additional predictive signal
Low	Implement data drift monitoring in production	Early detection of model degradation

File Inventory

File	Description
`London.csv`	Raw property listings dataset (3,480 rows)
`LondonHousePrice.ipynb`	Full EDA and model training workflow
`housing_price_model.pkl`	Serialised XGBoost production model
`app.py`	Flask API application
`requirements.txt`	Python package dependencies
`runtime.txt`	Python version specification for Heroku

Evaluation Metric Definitions

Metric	Interpretation
MAE	Average error in pounds; lower is better
RMSE	Error penalising large mistakes; lower is better
R^2 Score	Variance explained; closer to 1.0 is better

This report documents the complete development lifecycle of a machine learning system for predicting London residential property prices — from data collection through API deployment.